

Lab 7 addresses the direct communication and command/control workflow for the physical *myCobot 280*. In this lab, students move from kinematic computation toward practical robot operation by establishing communication with the robot, interpreting the exchanged data, sending motion commands, and checking how the real robot responds.

The lab is solved with Python in Jupyter notebooks and is grounded in the provided robot-control implementation used in the course environment. Before starting the practical work, inspect the serial communication path, the communication-frame handling, and the robot-control workflow in the codebase carefully. This will help you relate notebook actions to the actual messages exchanged with the *myCobot 280*, the robot states that are read back, and the safety checks that must be respected before motion is enabled.

Learning Objectives Serial Communication & Robot Control Lab

After completing this lab, students will understand how the *myCobot 280* is addressed through a practical serial communication workflow and how direct communication supports safe robot control in the laboratory. They will be able to distinguish between commanded and measured robot states, interpret the role of communication frames in reliable robot interaction, and use both position-oriented and velocity-oriented control modes with appropriate validation.

By the end of the lab, students will be able to:

- Explain the role of serial communication in the practical control workflow of the *myCobot 280*.
- Interpret the purpose of a communication frame or packet structure for robot commands and returned state data.
- Send joint-angle or position commands and read back encoder or measured joint-state information in the provided notebook workflow.
- Distinguish position mode from velocity mode and understand when each mode is appropriate.
- Compare commanded robot motion with measured robot motion and assess deviations in practical execution.
- Apply basic validation and safety-stop procedures before and during robot operation.

Short Questions

- 1) What is the purpose of serial communication in the practical control workflow of the *myCobot 280*?
- 2) Why is a communication frame or packet structure required when commands and state data are exchanged with the robot?
- 3) Explain the difference between commanded joint values and measured encoder or joint-state values.
- 4) What is the difference between position mode and velocity mode in practical robot operation?
- 5) Why must communication and motion commands be validated before they are sent to the real robot?
- 6) What is the purpose of a safety stop in a robot-control workflow?
- 7) Why is it important to compare commanded robot motion with measured robot motion during practical experiments?

- 8) Give practical reasons why direct communication with a real robot requires more caution than purely simulated motion execution.

Serial Communication & Robot Control

This lab is the first step in which students interact directly with the physical *myCobot 280* through the practical communication and control workflow used in the course environment. Up to this point, several labs focused on modeling, kinematics, or calibration. Here, the emphasis shifts to the question of how the robot is actually addressed, how commands are transmitted, how measured robot data is returned, and how this exchange must be checked before real motion is allowed.

The practical notebook contains the actual hands-on steps, command cells, and measurements for the lab. The purpose of this text is therefore not to reproduce the notebook, but to explain the basic workflow and the engineering considerations behind direct communication with the robot.

Why Direct Communication with the Real Robot Matters

Direct communication with the real robot is the point at which software assumptions are confronted with physical hardware behavior. A simulated robot usually follows commands exactly within the assumptions of the model. A real robot, by contrast, only moves after a communication channel has been opened successfully, a command has been encoded correctly, the internal controller has accepted the request, and the mechanical system has executed the motion.

For the *myCobot 280*, this means that practical robot control is not only a matter of choosing target values. It also requires understanding how these targets are transmitted to the robot, which quantities can be read back from the robot, and how to detect whether the executed motion corresponds to what was requested. This is why direct communication is a central step in a realistic robot-control workflow.

The Serial Communication Channel

In this lab, communication with the *myCobot 280* takes place through a serial connection. Conceptually, a serial channel is a byte stream between the computer and the robot controller. The notebook code opens this channel, configures it with the expected communication settings, writes command data to the robot, and reads back status or measurement data returned by the robot.

The important practical point is that the serial interface is the gateway to the physical system. If the wrong device is addressed, if the connection parameters are inconsistent, or if data is transmitted in an invalid form, the robot cannot respond reliably. The provided implementation is located in `serial_iface.py`, which opens the serial port (typically `/dev/serial0` at 1 000 000 baud) and exposes the low-level read/write interface used by the rest of the workflow. Students should therefore inspect the provided serial communication and robot-control workflow in the codebase before running the notebook,

so that the sequence from opening the communication channel to issuing a robot command is clear.

Communication Frames and Packet Structure

A byte stream alone is not sufficient for reliable robot communication. The controller must be able to distinguish where one message begins, where it ends, what kind of request is contained in it, and how the transmitted payload should be interpreted. For this reason, practical robot communication uses structured frames or packets.

At a conceptual level, a communication frame contains enough information to identify the command type, transport the associated payload, and support basic validity checks. In the *myCobot 280* workflow, this structure is what turns raw bytes into meaningful robot actions such as reading state information, sending a joint target, or stopping motion. Concretely, the *myCobot 280* uses frames of the form **FE FE LEN CMD [payload] FA**, where **FE FE** is the header, **LEN** counts the bytes from **CMD** through **FA** inclusive, **CMD** identifies the command type, and **FA** is the tail byte. The key command types used in this lab are read angles (0x20), send angles (0x22), stop (0x29), and resume (0x28). Students should pay attention to how the provided implementation assembles outgoing commands and how returned data is parsed again into usable robot-state information.

Without a frame structure, the controller could not interpret command boundaries reliably and the notebook could not decode returned state data in a robust way. The frame is therefore the practical basis of all subsequent robot-control actions in this lab.

Reading Robot State

After communication has been established, one of the first important operations is to read the robot state. In practical operation, state information may include encoder readings, measured joint values, or other quantities returned by the controller that describe the robot configuration at the current instant. In the provided implementation, the function `read_angles_deg()` sends a read-angles request and parses the response, where the joint angles are returned as six `int16` values, each representing degrees $\times 100$.

This distinction between commanded and measured values is essential. Commanded values are the targets sent from the notebook to the robot. Measured values are the states reported back by the robot after sensing and internal processing. These two need not be identical at every instant. Differences can arise because the robot is still moving toward the target, because communication is sampled at discrete times, because the controller applies limits internally, or because the mechanical system does not follow the request perfectly.

For this reason, reading robot state is not a secondary feature but a central part of closed-loop practical validation. Students should interpret returned joint or encoder data as the

reference for what the robot actually did, not merely as a confirmation that a command was issued.

Sending Position Commands

One common form of practical robot control is to send joint-angle or position commands. In this case, the notebook specifies a desired robot configuration or target pose and passes this information through the communication workflow to the *myCobot 280*. The controller then attempts to move the robot toward that target. The corresponding function `send_angles_deg()` encodes each joint angle as an `int16` (degrees $\times$ 100) together with a speed byte (1–100) inside the send-angles frame.

Before such a command is sent, it should be checked for plausibility. The intended target should be understood, the robot should have sufficient free space to move safely, and the requested motion should be consistent with the expected mode of operation. The fact that a target is numerically well-formed does not yet mean that it is safe for the real robot.

In the practical lab environment, students should use small, well-understood motions first and confirm the resulting state feedback before progressing to larger movements. This is particularly important in a first direct communication lab, because the central learning goal is not only to send commands, but to understand how the robot responds to them in practice.

Velocity Mode and Position Mode

The two most important operating ideas in this lab are position mode and velocity mode. In position mode, a target configuration or target position is specified, and the robot controller drives the robot toward that target. In velocity mode, the commanded quantity is instead a motion rate, so the robot moves as long as the velocity request remains active or until another condition stops it.

These two modes should not be treated as interchangeable. Position mode is usually appropriate when the desired end state is known in advance and the task is to reach that state in a controlled way. Velocity mode is useful when motion must be adjusted continuously, when incremental behavior is required, or when students want to observe how a commanded rate is reflected in the measured motion of the robot. In the code-base, the ROS 2 control node `mycobot_control.py` exposes these modes through the topics `/mycobot/joint_command` (position), `/mycobot/joint_velocity` (velocity), and `/mycobot/ee_pose` (Cartesian).

Because velocity commands can continue to produce motion as long as they remain effective, they require particular care. Practical supervision, limited command magnitudes, and immediate access to a stop mechanism are especially important when velocity mode is used on the physical *myCobot 280*.

Safety Stops and Practical Precautions

Direct robot communication always requires a safety-oriented mindset. Before motion is enabled, students should make sure that the workspace is clear, that no unintended contact can occur, that the motion direction is understood, and that a stopping procedure is known in advance. One should never issue robot commands first and consider safety only afterward.

A safety stop is the practical mechanism used to terminate or inhibit robot motion when an unexpected situation occurs. This may be necessary if the robot moves in the wrong direction, if the observed motion does not match the intended command, if communication behavior is unclear, or if a person or object enters the robot workspace unexpectedly.

In the notebook workflow, students should treat the stop functionality as an essential part of robot operation rather than as an exceptional feature. Safe experimentation means planning the stop path before starting the motion, keeping commanded motions modest during initial tests, and validating the communication result after each important step.

Validating Commanded and Measured Motion

An essential part of this lab is the comparison between commanded motion and measured motion. If a command requests a certain joint configuration, the returned encoder or joint-state data should be checked to see whether the robot actually approached that configuration. If a velocity is commanded, the measured state evolution should be inspected to see whether the observed direction and rate of motion are consistent with the request.

This comparison is important for several reasons. First, it reveals whether the communication chain is functioning correctly. Second, it helps detect interpretation errors, for example if a command is encoded differently from what was intended. Third, it provides a practical understanding of controller behavior, delays, and real hardware response. In short, the measured robot state is the evidence by which a command is validated.

For the *myCobot 280*, this means that successful robot control is not defined merely by transmitting commands, but by verifying that the real robot motion agrees sufficiently with the requested behavior. This validation step is one of the main learning outcomes of the lab and should accompany every practical communication experiment in the notebook.

Competences

After successfully completing the Serial Communication & Robot Control lab, students will have acquired the following competences:

- They understand how direct serial communication is used to operate the *myCobot 280* in the practical lab workflow.

- They can interpret the conceptual structure and purpose of robot communication frames and exchanged command/state data.
- They are able to send motion commands and read measured robot-state information in the provided Python/Jupyter environment.
- They can distinguish commanded values from measured values and use this comparison to judge execution quality.
- They understand the practical difference between position mode and velocity mode.
- They can apply elementary validation, safety precautions, and safety-stop handling during real robot operation.